

Decorating Our Parking Charges

Software Design & Programming

Object-Oriented Programming II

Joseph DiBiasi

University of Denver College of Professional Studies

5/8/2026

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, MS

Dean: Bobbie Kite, MLS

Introduction

Our initial implementation of creating parking strategies using a flexible series of individual smaller building blocks was a great success! The strategy pattern gave us a common method for all the strategy classes to implement calculations while the factory pattern gave us a central location for the actual creation of these strategies. The end result was a system for building complex strategies that was decoupled to allow for easy growth and maintenance with minimal disruption to other pieces of the application. Yet, we knew we could do more to improve our system organization and the readability of our code. Our focus this development cycle was the rework of our strategy pattern classes into the decorator pattern.

Requirements

Redesigning our flexible strategy pattern into a more modular decorator pattern required us adhering to some basic requirements to start. The first requirement was to rework our parking strategy classes into parking decorators implementing their original rate modifications. Next, we needed to transform our parking charge calculations from a method called in the transaction manager into a functional decorator workflow. To accommodate this workflow, a new parking charge calculator class needed to be created. This new factory had to be capable of not only creating new parking charge calculators but also wrapping them with our new decorator classes. Finally, the parking lot class had to be reworked to utilize the new parking charge calculator. These requirements could have been overwhelming to implement all

at once, but thankfully our initial parking strategy implementation utilized many similar features.

High-Level Design

The initial parking strategy implementation had very similar goals in creating a flexible and maintainable system. Similar goals and similar patterns can lead to similar designs. Our new decorator system was an excellent example of this as many of our new decorator classes were simply the old strategy classes with minimal functional changes besides a refactor of the class name to reflect the new decorator pattern. The main change in design came from how we were going to combine the new decorator classes to achieve the same buildable strategy pattern.

All decorator classes were still designed to implement a shared parking charge interface to ensure they would be implementing the same method for rate modifications, but there was a twist. Instead of the decorator classes implementing the parking charge interface directly, they would extend a base parking charge decorator class that implemented the parking charge interface. This base decorator class is abstract and so has the effect of being wrapped or 'decorated' by the more concrete decorator classes that implement the actual calculation method. This common parking charge interface has one more connection to it that lets us complete the decorator workflow; the initial rate calculation for parking charges. This method originally lived in the transaction manager, but it has been reworked into a concrete parking charge calculator class that implements the same shared parking charge interface.

The parking strategy factory has been converted to not only create a parking charge calculator but also wrap that calculator in as many different parking charge decorators as

desired. This allows parking lots to hide much of that rate calculation business logic by incorporating a single parking charge calculator class instead of multiple strategies. When the parking lot needs to calculate its rates, it simply asks the parking charge calculator to calculate its final rate. The base calculate method is called as well as all of the decorators calculate method that have wrapped it. This rate will be calculated regardless of how many different decorators wrap it because all of them override the same base parking charge interface method.

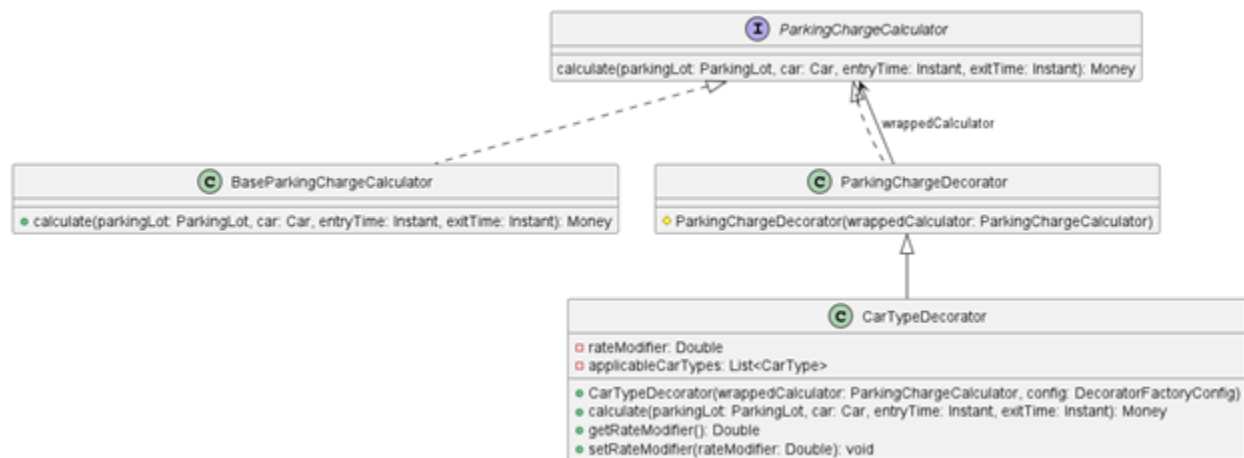


Figure 1. Decorator Pattern UML Diagram Regarding Rate Calculation Workflow

Implementation

This week's design cycle was an interesting one. Normally it is the actual implementation of the new code that is the challenging part. This week, the challenge was wrapping our heads around the idea of wrapping objects. A large amount of developer time was spent trying to figure out how to convert our existing strategy pattern logic into the decorator pattern. We felt that our initial design was close to the mark, yet when we tried diving into the work, we quickly got off track and were redesigning far more than intended. It was only after a

hard reboot on the initially attempted redesign work that we were able to make headway on implementing the decorator pattern.

Once that initial design hurdle was overcome, the actual work proceeded swiftly. As was alluded to in the high-level design portion, most of the decorator classes were a simple lift and shift from their strategy implementation. The redesign of the factory was much the same, though we did break out the validation portion into its own method as this increased the readability of the new decorator process while maintaining the same functionality. Finally, the transaction manager was modified by lifting the base charge calculation method into its new role as the calculator class that gets decorated. Once the workflow redesign was implemented, we did the same redesign process with our test cases to ensure that the logic remained sound. Most test work was a simple case of lift and shifting, similar to the main classes, but there were new areas to cover as well as reworks to accommodate the new decorator workflow.

Summary

If the initial strategy and factory implementation felt like a lesson in following good design practices from the start, the decorator pattern redesign was a lesson in measuring twice and cutting once. Understanding the logic behind the design was the challenge of this development cycle. We had to dive into implementing the decorator pattern in the wrong direction before realizing how it connected with our existing structure. Do not be afraid to go back to the drawing board; choosing the right design for the right challenge is worth the effort. In our case, it led to us achieving a more modular design pattern for creating our parking rate strategies while still staying true to our original structure.

References

Shvets, Alexander. n.d. "Structural Design Patterns." Refactoring.Guru. Accessed May 9, 2026.

<https://refactoring.guru/design-patterns/structural-patterns>.